المدرسة العليا للتكنولوجيا - كلميم

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵓⵙⵙⵏⴰⵡⵉⵜ - ⴳⵍⵎⵉⵎ

ÉCOLE SUPÉRIEURE DE TECHNOLOGIE - GUELMIM

Mini-Projet :

Système intelligent de prédiction du succès des produits e-commerce

Filière : B-SDSI S6

Module : Système intelligents

Réalisées par :

BEN ZHIR Wafa

IKSOD Salma

Encadré par :

Pf. AIT BAHA Tarek

Année universitaire : 2025-2026

Chapitre 1 : Introduction et présentation du projet :

I- Contexte général :

Le commerce électronique (e-commerce) est devenu l'un des secteurs les plus compétitifs de l'économie numérique. La satisfaction client et la popularité des produits sont des indicateurs clés de succès pour toute plateforme de vente en ligne. Dans ce contexte, la capacité à anticiper si un produit sera recommandé ou non par les clients représente un avantage concurrentiel majeur.

Ce projet s'inscrit dans cette problématique en appliquant des techniques de Machine Learning supervisées pour prédire automatiquement la popularité d'un produit à partir des avis clients.

II- Dataset utilisé :

Le projet utilise le dataset « Women's E-Commerce Clothing Reviews », disponible sur Kaggle. Ce dataset contient des avis laissés par des clientes sur des vêtements vendus sur une plateforme e-commerce.

Il rassemble 23 486 lignes et 11 colonnes couvrant trois types de données :

- Données numériques : Age, Rating (note de 1 à 5), Positive Feedback Count
- Données catégorielles : Division Name, Department Name, Class Name
- Données textuelles : Title (titre de l'avis), Review Text (contenu de l'avis)

Colonne	Description	Type
Clothing ID	Identifiant unique du produit	Numérique
Age	Âge de la cliente ayant laissé l'avis	Numérique
Title	Titre de l'avis client	Texte
Review Text	Texte complet du commentaire	Texte
Rating	Note du produit (1 à 5)	Numérique
Recommended IND	Variable cible : 1 = recommandé, 0 = non	Binaire
Positive Feedback Count	Nombre de feedbacks positifs reçus	Numérique
Division Name	Catégorie principale du produit	Catégorielle
Department Name	Département du produit	Catégorielle
Class Name	Sous-catégorie du produit	Catégorielle

III- Objectif du projet :

L'objectif principal est de construire un modèle de classification binaire capable de prédire si un produit sera recommandé ou non par les clientes, à partir des informations disponibles dans le dataset.

Plus précisément, le projet vise à :

- Comprendre les données clients et les facteurs influençant la recommandation
- Analyser les avis textuels pour extraire des informations de sentiment
- Construire et comparer plusieurs modèles de Machine Learning
- Déployer une application interactive Streamlit pour prédire en temps réel

IV- Type de problème et variable cible :

Il s'agit d'un problème de classification binaire. La variable cible est Recommended IND :

- Valeur 1 : la cliente recommande le produit
- Valeur 0 : la cliente ne recommande pas le produit

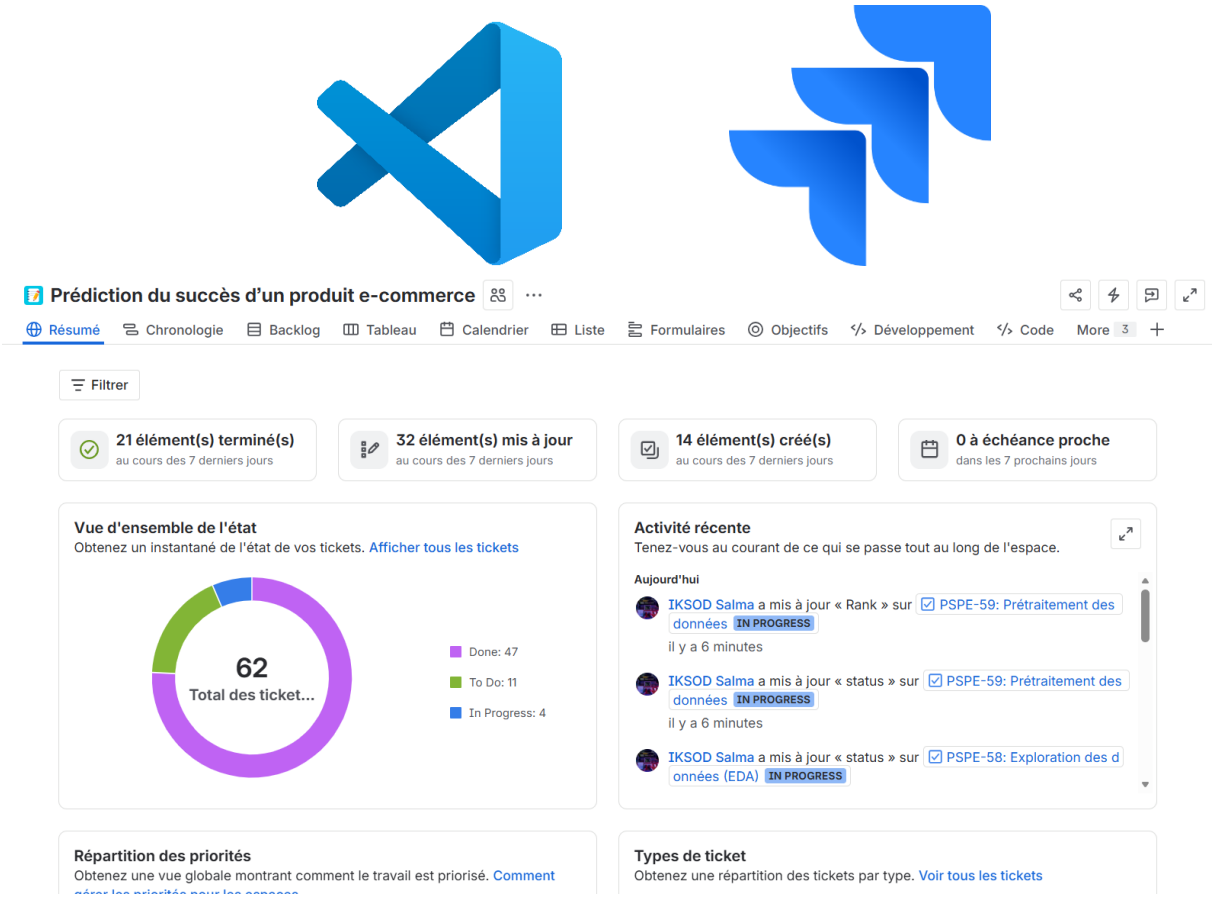
Le dataset présente un déséquilibre notable : 82,6 % des avis correspondent à des produits recommandés (classe 1), contre seulement 17,4 % pour la classe 0. Ce déséquilibre a été pris en compte tout au long du projet.

Chapitre 2 : Les outils et les technologie utilisés :

I- Environnement de développement :

Le projet a été développé en **Python** avec **Jupyter Notebook** pour l'analyse et **Streamlit** pour l'application.

Visual Studio Code (**VS Code**) a été utilisé comme environnement de développement pour sa simplicité et ses fonctionnalités. **Jira** a permis de gérer les tâches et suivre l'avancement du projet.



Le rapport a été rédigé avec un outil **de traitement de texte**, tandis que **GitHub** a été utilisé pour le versioning et la collaboration. Lien de **GitHub** [BenWafae/womens-ecommerce-product-success: Mini-projet SI - Prédiction du succès d'un produit e-commerce à partir du dataset Women's E-Commerce Clothing Reviews. Projet de classification binaire avec exploration, preprocessing, modèles ML et interface Streamlit.]

II- Bibliothèque python utilisées :

- pandas : chargement, nettoyage, filtrage et transformation du dataset.
- numpy : calculs numériques et manipulation de tableaux.
- matplotlib : création de graphiques statiques (histogrammes, barplots, etc.)
- seaborn : visualisations statistiques enrichies (heatmaps, boxplots, countplots)

Scikit-learn est la bibliothèque centrale du projet. Elle a été utilisée pour :

- train_test_split : séparation des données en ensembles d'entraînement et de test
- StandardScaler, OneHotEncoder : prétraitement des données

- LogisticRegression, DecisionTreeClassifier, RandomForestClassifier : modèles
- GridSearchCV, RandomizedSearchCV : optimisation des hyperparamètres
- cross_val_score : évaluation robuste par validation croisée
- accuracy_score, f1_score, confusion_matrix, classification_report : métriques
- TextBlob : analyse de sentiment — extraction de la polarité et de la subjectivité des avis
- WordCloud : visualisation des mots les plus fréquents dans les avis clients
- TfidfVectorizer (scikit-learn) : transformation des textes en vecteurs numériques pour l'application Streamlit
- SHAP (SHapley Additive exPlanations) : interprétation globale et locale des prédictions du modèle Random Forest
- LIME (Local Interpretable Model-agnostic Explanations) : explication locale d'une prédiction individuelle
- joblib / pickle : sérialisation des modèles entraînés pour réutilisation dans Streamlit
- Streamlit : déploiement d'une interface web interactive pour la prédiction en temps réel

Bibliothèque	Rôle	Catégorie
pandas	Manipulation du dataset	Data
numpy	Calculs numériques	Data
matplotlib / seaborn	Visualisations graphiques	Viz
scikit-learn	Modèles ML, prétraitement, métriques	ML
TextBlob	Analyse de sentiment NLP	NLP
SHAP / LIME	Explicabilité IA	XAI
Streamlit	Application web interactive	Déploiement
pickle / joblib	Sauvegarde des modèles	Déploiement

Chapitre 3 : Exploration et analyse des données (EDA):

I- Informations générales sur le dataset :

L'exploration a débuté par un examen des caractéristiques générales du dataset. Les commandes df.shape, df.info() et df.describe() ont permis d'obtenir :

- 23 486 lignes et 11 colonnes avant nettoyage
- Variables numériques : Age, Rating, Positive Feedback Count, Clothing ID
- Variables catégorielles : Division Name, Department Name, Class Name
- Variables textuelles : Title, Review Text
- Variable cible binaire : Recommended IND (0 ou 1)

II- Analyse de la variable cible :

Le countplot de la variable Recommended IND révèle un déséquilibre marqué :

- Classe 1 (recommandé) : 82,6 % des observations
- Classe 0 (non recommandé) : 17,4 % des observations

Ce déséquilibre est crucial : utiliser uniquement l'accuracy comme métrique serait trompeur. Le F1-score et le recall sur la classe minoritaire ont été privilégiés pour l'évaluation des modèles.

III- Analyse des valeurs manquantes, des outliers et des corrélations :

Les données présentent des valeurs manquantes surtout dans *Title* et *Review Text*, mais elles restent globalement dispersées et peu nombreuses dans les variables numériques. Les outliers ont été traités avec la méthode **IQR** pour *Age* et *Positive Feedback Count*, tandis que *Clothing ID* a été conservé. Enfin, l'analyse de corrélation montre que *Rating* est fortement lié à ***Recommended IND***, contrairement aux autres variables qui ont un impact faible.

IV- Analyse de la polarité du sentiment :

La distribution de la polarité (calculée avec TextBlob après le Feature Engineering) a été comparée entre les deux classes. Les produits recommandés (classe 1) présentent une distribution décalée vers des polarités positives (0,3 à 0,5), tandis que les non recommandés (classe 0) se concentrent autour de polarités neutres (0,0 à 0,2). Cela confirme que la polarité du texte a un pouvoir prédictif réel.

Chapitre 4 : Prétraitement et feature engineering :

I- Prétraitement des données (nettoyage et outliers):

Le nettoyage des données a consisté à supprimer les lignes où ***Review Text*** est manquant, remplacer les valeurs absentes de *Title* par « **No Title** » et celles des variables catégorielles (***Division Name***, ***Department Name***, ***Class Name***) par « **Unknown** », ce qui a permis d'éliminer complètement les valeurs manquantes. Ensuite, les outliers ont été traités avec la méthode **IQR** sur ***Age* et *Positive Feedback Count*** en supprimant les valeurs hors des bornes définies, et la colonne inutile *Unnamed: 0* a été supprimée afin d'alléger le dataset.

II- Feature engineering et préparation du modèle :

Trois nouvelles variables ont été créées à partir du texte (***review_length***, ***polarity***, ***subjectivity***) pour enrichir le modèle et capturer des informations pertinentes. Les variables catégorielles ont été transformées par **One-Hot Encoding** pour être exploitables par les modèles, tandis que les variables numériques ont été normalisées avec **StandardScaler** afin d'éviter les biais liés aux différences d'échelle. Enfin, les données ont été divisées en ensembles d'entraînement (80 %) et de test (20 %) avec une stratification garantissant une distribution équilibrée des classes pour une évaluation fiable du modèle.

Chapitre 5 : Modélisation et optimisation:

I- Choix des modèles:

Le problème est une classification binaire avec : données de taille moyenne (20 000), déséquilibre des classes (82/18,) variables mixtes (numériques + catégorielles + texte)

Trois modèles complémentaires ont été choisis : **Régression Logistique** : baseline simple et interprétable. **Decision Tree** : capture des règles (ex : Rating ≥ 4) **Random Forest** : plus robuste et performant.

II- Résultats initiaux :

- **Régression Logistique** : meilleure baseline (F1 = 0,96 / classe 1)
- **Decision Tree** : moins performant → overfitting
- **Random Forest** : très bon compromis (meilleure précision)

Problème principal : performance plus faible sur la **classe minoritaire (classe 0)**

III- Gestion du déséquilibre :

- Utilisation de `class_weight='balanced'`
- Impact **faible amélioration**
  Le **stratify** avait déjà bien corrigé le déséquilibre.

IV- Validation croisée (CV=3) :

Une validation croisée à 3 folds a été réalisée sur les trois modèles pour valider la robustesse des résultats et s'assurer qu'ils ne sont pas dus au hasard d'un seul split :

Modèle	F1 moyen (cv=3)	Écart-type	Interprétation
Random Forest	0,9601	0,0017	Meilleure moyenne
Logistic Regression	0,9597	0,0005	Plus stable
Decision Tree	0,9490	0,0012	Léger overfitting

*La différence entre **Random Forest** et Régression Logistique est négligeable (0,0004 de F1). Le Random Forest est retenu comme modèle final pour sa robustesse et sa capacité à fournir des informations d'explicabilité via SHAP.*

V- Optimisation des modèles :

- **Régression Logistique** : amélioration très légère
- **Decision Tree** : gros gain après limitation de profondeur (corrige overfitting)
- **Random Forest** : meilleur modèle après optimisation

Meilleurs paramètres RF : n_estimators=100 , max_depth=10 , max_features=log2

VI- Comparaison finale :

- **Random Forest optimisé = meilleur modèle**
  - Accuracy ≈ 93,75 %
  - F1 ≈ 0,9616

- Les trois modèles sont proches, mais RF est le plus robuste

VII- Explicabilité avec SHAP :

- **Variable la plus importante : Rating (très dominant)**
- Ensuite :
  - polarity (sentiment)
  - subjectivity

Insights clés :

- Rating élevé → recommandation
- Rating faible → non recommandé
- Variables catégorielles → impact faible

SHAP permet :

- explication globale (importance des variables)
- explication locale (prédictions individuelles)

VIII-Conclusion :

Modèle final : Random Forest optimisé

Très bonnes performances :

- F1 = 0,9616
- Accuracy = 93,75 %

Interprétable grâce à SHAP , Le **Rating + analyse de sentiment** sont les facteurs les plus déterminants

Chapitre 6 : Évaluation des modèles :

I- Métriques d'évaluation utilisées :

Pour évaluer les performances de chaque modèle de manière rigoureuse, quatre métriques complémentaires ont été utilisées. En raison du déséquilibre des classes (82,6 % / 17,4 %), l'accuracy seule ne suffit pas — le F1-score est la métrique principale.

Étrique	Définition	Intérêt dans ce projet
Accuracy	Proportion de bonnes prédictions sur le total	Indicateur global, mais insuffisant seul
Precision	Parmi les prédits positifs, ceux réellement positifs	Limite les fausses alertes (FP)

Recall	Parmi les vrais positifs, ceux bien détectés	Crucial pour la classe minoritaire (cl. 0)
F1-Score	Moyenne harmonique de Precision et Recall	Métrique principale ici

II- Résultats par modèle :

1- Régression Logistique :

Accuracy : 93,46 % — Precision : 97,03 % — Recall : 94,98 % — F1 (cl. 1) : 96,00 % — F1 (cl. 0) : 82 %

La Régression Logistique constitue une baseline très solide. Elle gère bien la classe majoritaire (F1 = 0,96) mais détecte moins bien la classe minoritaire « Non recommandé » (F1 = 0,82). La matrice de confusion montre peu de faux positifs mais davantage de faux négatifs.

2- Decision Tree :

Accuracy : 92,18 % — Precision : 95,52 % — Recall : 94,98 % — F1 (cl. 1) : 95,25 % — F1 (cl. 0) : 78 %

Le Decision Tree sans contrainte de profondeur présente des signes d'overfitting : son F1 sur la classe minoritaire chute à 0,78, inférieur à la Régression Logistique. Après optimisation (max_depth=5, criterion=entropy), il remonte à F1 = 0,9597, corrigeant en grande partie le problème.

L'amélioration de +0,0072 de F1 après GridSearchCV confirme que la contrainte de profondeur est essentielle pour le Decision Tree sur ce dataset.

3- Random Forest (modèle final) :

Accuracy : 93,75 % — Precision : 97,19 % — Recall : 94,51 % — F1 (cl. 1) : 96,16 % — F1 (cl. 0) : 82 %

Le Random Forest optimisé (RandomizedSearchCV) obtient les meilleures performances globales. Il présente la meilleure précision (97,19 %) et le meilleur F1 moyen en cross-validation (96,01 %). Sa matrice de confusion montre : 3 311 vrais positifs, 621 vrais négatifs, 175 faux négatifs et 92 faux positifs.

III- Comparaison finale :

Modèle	Accuracy	Precision	Recall	F1 (cl.1)	F1 (cl.0)	CV F1
Logistic Reg.	93,5 %	97,0 %	94,9 %	96,0 %	82 %	95,97
Decision Tree	92,2 %	95,5 %	94,9 %	95,2 %	78 %	94,90
❖ Random Forest	93,7 %	97,2 %	94,5 %	96,2 %	82 %	96,01

La différence entre Random Forest et Régression Logistique est très faible (0,0004 de F1). Le Random Forest est retenu pour sa robustesse et sa capacité à fournir l'importance des variables via SHAP.

IV- Cross-validation :

Une validation croisée à 3 folds a été réalisée pour confirmer que les résultats ne sont pas issus d'un simple hasard de split. La méthodologie cross_val_score de scikit-learn évalue chaque modèle sur 3 partitions indépendantes et retourne la moyenne et l'écart-type du F1.

- Random Forest : F1 moyen = 0,9601, écart-type = 0,0017 — meilleure moyenne
- Régression Logistique : F1 moyen = 0,9597, écart-type = 0,0005 — modèle le plus stable
- Decision Tree : F1 moyen = 0,9490, écart-type = 0,0012 — léger overfitting confirmé

La stabilité de la Régression Logistique (plus faible écart-type) est remarquable, mais le Random Forest conserve l'avantage en performance moyenne et en capacité d'interprétabilité.

V- Impact de class_weight='balanced':

Les trois modèles ont été réentraînés avec le paramètre class_weight='balanced'. L'amélioration observée est marginale : +0,014 sur le F1 de la classe 0 pour la Régression Logistique, quasi nulle pour les deux autres. Cela confirme que le paramètre stratify=y utilisé lors de la séparation train/test avait déjà correctement compensé le déséquilibre.

Chapitre 7 : Application Streamlit :

I- Objectif de l'application « ShopSense AI »:

L'application **ShopSense AI** a pour objectif de fournir une **plateforme web interactive** permettant de **prédire en temps réel le succès d'un produit e-commerce** à partir de données clients (rating, avis, caractéristiques produit), tout en offrant une **visualisation complète des données (EDA)**, une **comparaison des modèles de Machine Learning**, et une **explication transparente des prédictions grâce à SHAP**, afin d'aider à la **prise de décision business basée sur l'intelligence artificielle.**

Chapitre 8 : Conclusion :

II- Bilan du projet :

Ce projet a permis de construire un pipeline complet de Machine Learning appliqué au e-commerce, depuis l'exploration des données jusqu'au déploiement d'une application interactive. L'objectif principal prédire si un produit sera recommandé ou non par les clientes a été atteint avec un niveau de performance élevé.

Le modèle final retenu est le **Random Forest** optimisé avec les paramètres suivants : n_estimators=100, max_depth=10, max_features='log2'. Il **atteint** une **accuracy** de 93,75 %, un **F1-Score** de 96,16 % sur la classe recommandée et un F1 moyen en cross-validation de 96,01 % sur 3 folds.